KIẾN TRÚC BỘ NHỚ NHẬN THỨC • V02 SPECIFICATION

HỆ CON MEMORY CỦA ERISTUTOR

Luồng Hoạt Động Toàn Diện & Phân Tích Kiến Trúc Sâu

Từ phát ngôn của người dùng đến ký ức bền vững và cơ chế tái hiện nhận thức

Dựa trên rà soát 105 module thực tế tại thư mục eris/src/memory/ (~29.504 dòng mã nguồn)

Mã tài liệu:	ERISTUTOR_20261107_V02_M02_MEMORY_FLOW_VI
Phiên bản:	V02 - Ngày 07/11/2026 (Múi giờ UTC+07:00)
Nguồn phân tích:	eris/src/memory/ (105 files, Subject Core, Living User, Understanding, Stores)
Phạm vi & Thẩm quyền:	Trích xuất & diễn giải cấu trúc mã nguồn; Chế độ CHỈ ĐỌC, bảo toàn logic.
Bộ điều phối chính:	MemoryKernel facade, MemoryCognitionCore, MemoryCommitPolicy, MemoryScorer

TỔNG QUAN TÀI LIỆU (EXECUTIVE SUMMARY)

Hệ con Memory của ErisTutor là kiến trúc nhận thức (Cognitive Memory Subsystem) được thiết kế đặc biệt để phục vụ gia sư ảo thông minh. Khác biệt với các hệ thống AI thông thường chỉ lưu log chat dạng phẳng hoặc nhồi nhét ngữ cảnh vào LLM, Eris Memory xây dựng một đường ống xử lý có kiểm soát cao:

1. Kiểm soát chiều Ghi (Ingest): Đọc hiểu ngữ nghĩa (MeaningFrame), xác minh nguồn gốc (Provenance), áp dụng chính sách commit 3 tầng khắt khe và tách bạch đề xuất (plan_ingest) khỏi việc ghi đĩa (commit_plan).
2. Chuyên biệt hóa lưu trữ: Phân bổ vào 9 kho lưu trữ chuyên biệt (Episodic, Semantic, Procedural, Relational, Repair, Residue, Candidate, Ledger, Canonical) với vòng đời và mức độ đào thải riêng biệt.
3. Kiểm soát chiều Đọc (Recall): Cơ chế trả lời sự thật xác định (answer()) chạy qua Bộ Ba Cổng kiểm duyệt nghiêm ngặt (zero LLM, zero hallucination); trong khi cơ chế gợi nhớ nhận thức (build_packet()) sàng lọc tư cách qua Cổng Nhập Viện (Admission Gate) để ngăn chặn rác transient/audit noise đi vào prompt.
4. Bảo trì nền (Maintain): Chuyển hóa candidate thành ký ức dài hạn, giải quyết mâu thuẫn và đồng bộ hóa chân dung người dùng sống (Living User) trong các pha idle/reflection.

CÁC TRỤ CỘT KIẾN TRÚC TRỌNG TÂM ĐƯỢC LÀM RÕ TRONG TÀI LIỆU:

CHIỀU GHI (INGEST) | 9 KHO CHUYÊN BIỆT | CHIỀU ĐỌC (RECALL) | BẢO TRÌ NỀN (MAINTAIN) | 8 BẤT BIẾN CỐT LÕI

[*] NGUYÊN TẮC THIẾT KẾ XUYÊN SUỐT CỦA ERISTUTOR

Mục tiêu tối thượng của hệ con Memory là: 'Nhớ ít mà chắc' thay vì 'nhớ nhiều mà bịa'. Mọi đường tắt có thể khiến mô hình tự tin nhớ sai đều bị chặn bằng một cổng deterministic có thể kiểm thử.

1. MEMORY LÀ GÌ VÀ KHÔNG PHẢI LÀ GÌ

Memory là hệ con bộ nhớ nhận thức của Eris AI. Nó không phải là một log hội thoại đơn thuần, cũng không phải là một file JSON chứa mọi phát ngôn thô của người dùng. Bản chất của Memory là một đường ống (cognitive pipeline) chuẩn hóa: biến tín hiệu thô thành ký ức có cấu trúc, có nguồn gốc rõ ràng (provenance) và có mức độ tin cậy được định lượng, rồi truy hồi lại một cách kiểm soát chặt chẽ để đưa vào nhận thức hoặc phát ngôn chính thức.

Bốn Động Từ Công Khai của Facade MemoryKernel

Runtime của ErisTutor chỉ tương tác với hệ thống bộ nhớ thông qua đúng 4 động từ công khai. Mọi tầng lưu trữ (stores), chính sách (policies) và bộ thực thi (executors) bên dưới đều bị giấu kín:

Động từ	Vai trò kiến trúc	Đặc tính luồng	Mục tiêu & Cam kết
ingest()	Tiếp nhận văn bản / evidence	Hot path & Async	Đọc nghĩa, gán provenance, tạo commit plan.
answer()	Trả lời câu hỏi sự thật trí nhớ	Deterministic (Zero LLM)	3 cổng bảo vệ: quyền, confidence >= 0.60, ambiguity <= 0.78.
build_packet()	Dựng context cho cognition	Tạo bias nội bộ	Tính điểm Relevance/Prior, lọc qua Cổng Nhập Viện (Admissio
maintain()	Bảo trì, dọn dẹp, chưng cất	Chạy nền (Idle/Dream)	Consolidate candidate, prune expired, liên kết mâu thuẫn.

Ba Ranh Giới Cứng (Architectural Invariants)

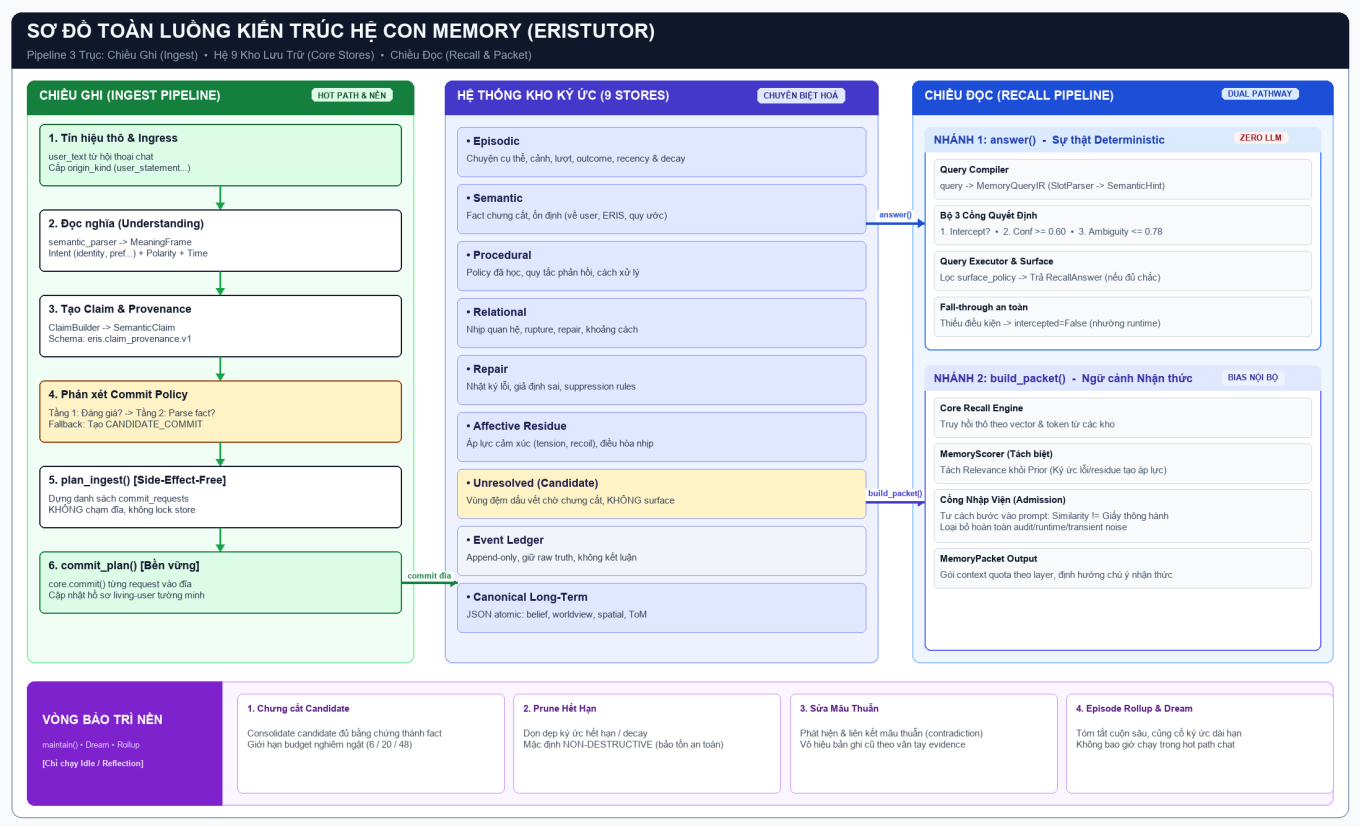
• **Mô hình chỉ được đọc nghĩa sơ bộ:** LLM/parser CHỈ được giải mã ngữ nghĩa ban đầu ở chế độ semantic_digest. Mọi quyết định về ontology, entailment, tính đáng nhớ (worthiness), thời gian lưu (retention) và lệnh commit đều do code thuần quyết định.

• **Nguyên tắc Single-Store-Commit:** Không có cửa sau hoặc store thứ hai: Mọi hành vi làm thay đổi trạng thái ký ức bắt buộc phải đóng gói thành StateCommitRequest và được thực thi duy nhất qua MemoryCognitionCore.commit().

• **Phân định Candidate và Fact:** Thông tin đáng nhớ nhưng chưa parse được thành cấu trúc rõ ràng TUYỆT ĐỐI KHÔNG được xem là 'đã nhớ chắc'. Chúng được lưu tạm vào vùng đệm Unresolved Memory dưới dạng candidate chờ chu trình chưng cất nền.

2. SƠ ĐỒ TOÀN LUỒNG KIẾN TRÚC (END-TO-END PIPELINE)

Sơ đồ dưới đây thể hiện toàn bộ chu trình xử lý của Eris Memory với 3 luồng chính: Chiều GHI (Ingest) chuyển hóa phát ngôn người dùng xuống các kho lưu trữ; Chiều ĐỌC (Recall) truy hồi chọn lọc lên nhận thức; và Vòng BẢO TRÌ NỀN (Maintain) chưng cất, củng cố dài hạn:



3. CHIỀU GHI: TỪ PHÁT NGÔN ĐẾN KÝ ỨC BỀN VỮNG

3.1 Understanding & MeaningFrame - Đọc nghĩa trước khi trích xuất

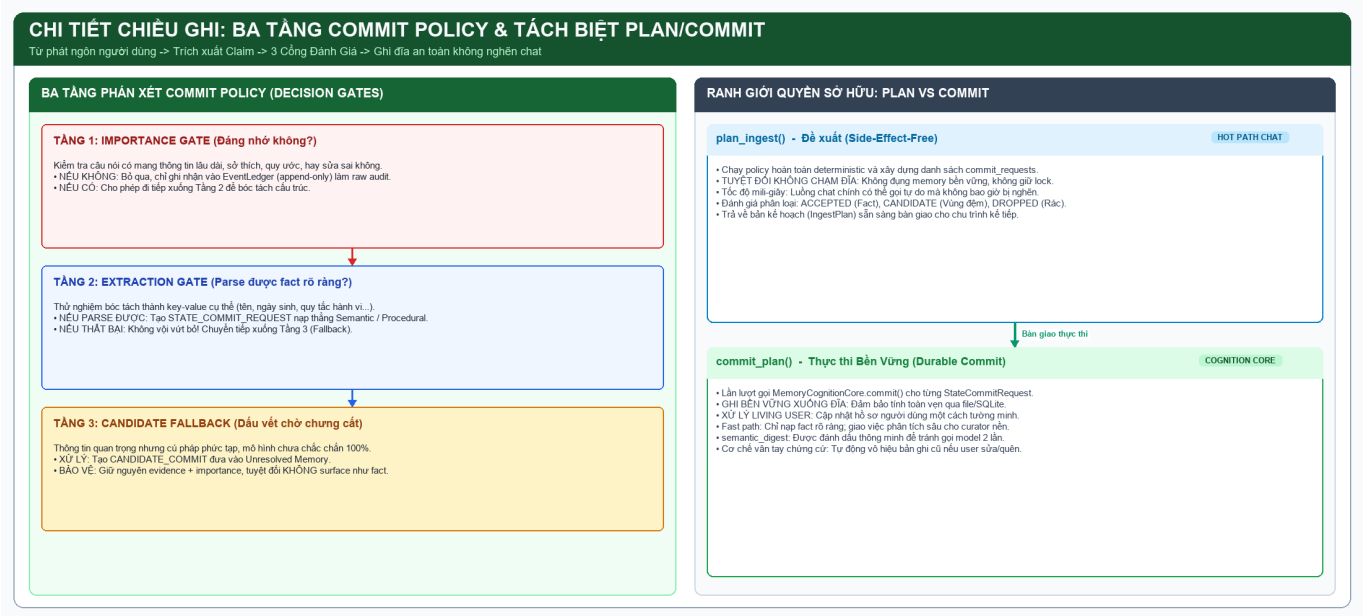
Thay vì ánh xạ thẳng văn bản người dùng thành vị từ (predicate) một cách máy móc, hệ thống phân tích câu nói thành MeaningFrame nhằm nắm bắt bản chất hành vi hội thoại: người dùng đang tự khai báo danh tính (identity), bày tỏ sở thích (preference), ra quy ước hướng dẫn (instruction), góp ý phản hồi (feedback), yêu cầu ghi nhớ (memory_request), hay chỉ chào hỏi xã giao (session_event)?

Mỗi Frame mang thêm hai thuộc tính then chốt: Polarity (asserted / denied / uncertain / corrected) và Time Scope (current / stable / past / habitual). Tầng parsing có 2 chế độ: Model GGUF (trả JSON frames rồi verify) và Rule Fallback (nhẹ, nhanh, cố tình đặt confidence vừa phải để không tự tin thái quá khi chưa gắn mô hình).

3.2 Claim & Provenance - Xác lập nguồn gốc dữ liệu

ClaimBuilder chuyển đổi MeaningFrame đã xác minh thành SemanticClaim. Mỗi claim mang thông tin provenance theo chuẩn eris.claim_provenance.v1, xác định rõ origin_kind thuộc một trong các loại: user_statement, assistant_utterance, observer_summary, runtime_state, fiction, hoặc unknown.

- **Quyền Ingress cấp nguồn:** Metadata nguồn gốc hoàn toàn do RUNTIME INGRESS cấp, không bao giờ để mô hình tự khai báo.
- **Đặc quyền của user_statement:** Chỉ phát ngôn trực tiếp từ người dùng (user_statement) mới được phép đưa vào Living User Profile; tóm tắt của observer hay lời văn của assistant tuyệt đối không được đi ké làm chứng cứ gốc.



3.3 plan_ingest() vs commit_plan() & Bảng Cấu Hình Ngưỡng

Đây là ranh giới quyền sở hữu quan trọng nhất của chiều ghi. plan_ingest() chạy hoàn toàn trong bộ nhớ, không chạm đĩa, cho phép luồng chat chính gọi tự do với độ trễ mili-giây. commit_plan() sau đó mới thực thi ghi đĩa bền vững và cập nhật hồ sơ người dùng một cách tường minh.

Tham số cấu hình (MemoryKernelConfig)	Giá trị ngưỡng	Ý nghĩa kiến trúc & Ràng buộc
answer_min_confidence	0.60	Ngưỡng tin cậy tối thiểu để answer() được phép trả lời trực tiếp.
answer_max_ambiguity	0.78	Ngưỡng mơ hồ tối đa: nếu câu hỏi mập mờ vượt quá 0.78, từ chối intercept.
digest_commit_min_confidence	0.62	Ngưỡng tin cậy khi digest nền; nếu thiếu chắc chắn -> rớt về candidate.
candidate_limit (small / normal / deep)	6 / 20 / 48	Hạn ngạch số lượng candidate chưng cất theo từng mức ngân sách tài nguyên.

Ba Chế Độ Ingest Chuyên Biệt

- **fast mode:** Đường nóng trực tiếp trong phiên chat. Tuyệt đối không gọi LLM; chỉ dùng rule/policy giá rẻ để bắt fact hiển nhiên (tên, ngày sinh, văn phong). Không đụng vào Living User JSON.
- **semantic_digest mode:** Đường nền bất đồng bộ. Có thể dùng mô hình phân rã ngữ nghĩa chuyên sâu khi hệ thống rảnh rỗi (idle) hoặc chờ phản hồi API.
- **reflection mode:** Đường ngẫm sâu trong chu trình kết thúc phiên (session end / dream) để chưng cất candidate và giải quyết mâu thuẫn. Không chạy sau mỗi lượt chat.

4. HỆ THỐNG 9 KHO KÝ ỨC CHUYÊN BIỆT (MEMORY STORES)

Nhận thức của ErisTutor phân tách rạch ròi thành 9 kho lưu trữ chuyên biệt. Mỗi kho phục vụ một mục đích riêng biệt với cấu trúc dữ liệu, tiêu chí đánh giá và chính sách đào thải (decay) độc lập:

1. Episodic Memory (Ký ức sự kiện)
Lưu lại các biến cố cụ thể đã diễn ra: bối cảnh (scene), lượt hội thoại, hành vi, bằng chứng và kết quả (outcome). Truy hồi dựa trên chuẩn hóa token, so khớp cụm từ/bigram, trọng số trường dữ liệu, độ nổi bật (salience), tính gần đây (recency) và hệ số phân rã (decay) - tuyệt đối không cộng điểm theo kiểu đếm token thô.

2. Semantic Memory (Ký ức ngữ nghĩa / Fact)
Chứa các sự thật đã được chưng cất và đạt trạng thái ổn định lâu dài (thông tin về người dùng, nhân vật ERIS, dự án, tri thức thế giới, quy ước làm việc). Cố tình KHÔNG dùng làm event log - sự kiện lẻ tẻ phải lưu tại EventLedger.

3. Procedural Memory (Ký ức quy trình & Hành vi)
Lưu các chính sách hành vi đã học được qua quá trình tương tác. Ví dụ: khi người dùng sửa hiểu lầm thì phải trả lời thẳng vào trọng tâm trước; khi bằng chứng thị giác yếu thì không được tự nhận là nhìn thấy màn hình. Đây là cách ứng xử, không phải fact.

4. Relational Memory (Ký ức quan hệ)
Ghi nhận nhịp điệu tương tác thực tế có bằng chứng: các pha xung đột (rupture), hàn gắn (repair), khoảng cách tâm lý và kiểu phản hồi ưa thích. Đây là dữ liệu khách quan, tuyệt đối không phải nơi suy diễn cảm tính dạng 'user is sad'.

5. Repair Memory (Ký ức sửa lỗi)
Lưu vết chi tiết về loại sai lầm đã xảy ra, giả định sai của mô hình, chứng cứ phản bác, chiến lược sửa chữa và các quy tắc ức chế (suppression rules) trong tương lai - nhằm ngăn chặn ERIS lặp lại chính sai lầm đó.

6. Affective Residue Memory (Dư âm cảm xúc)
Lưu lại các dư chấn tâm lý dưới dạng áp lực nội bộ (tension, attachment, recoil, unfinished loop). Áp lực này tác động trực tiếp lên nhịp nói, khoảng lặng và mức độ thận trọng, TUYỆT ĐỐI KHÔNG tự động biến thành câu thoại phát ngôn.

7. Unresolved Memory (Candidate - Vùng đệm)
Vùng đệm giữa 'người dùng bảo nhớ' và 'được phép công nhận là đã nhớ chắc'. Candidate chỉ là dấu vết tiềm năng chứa evidence và importance, tuyệt đối không bao giờ được surface ra ngoài như một sự thật.

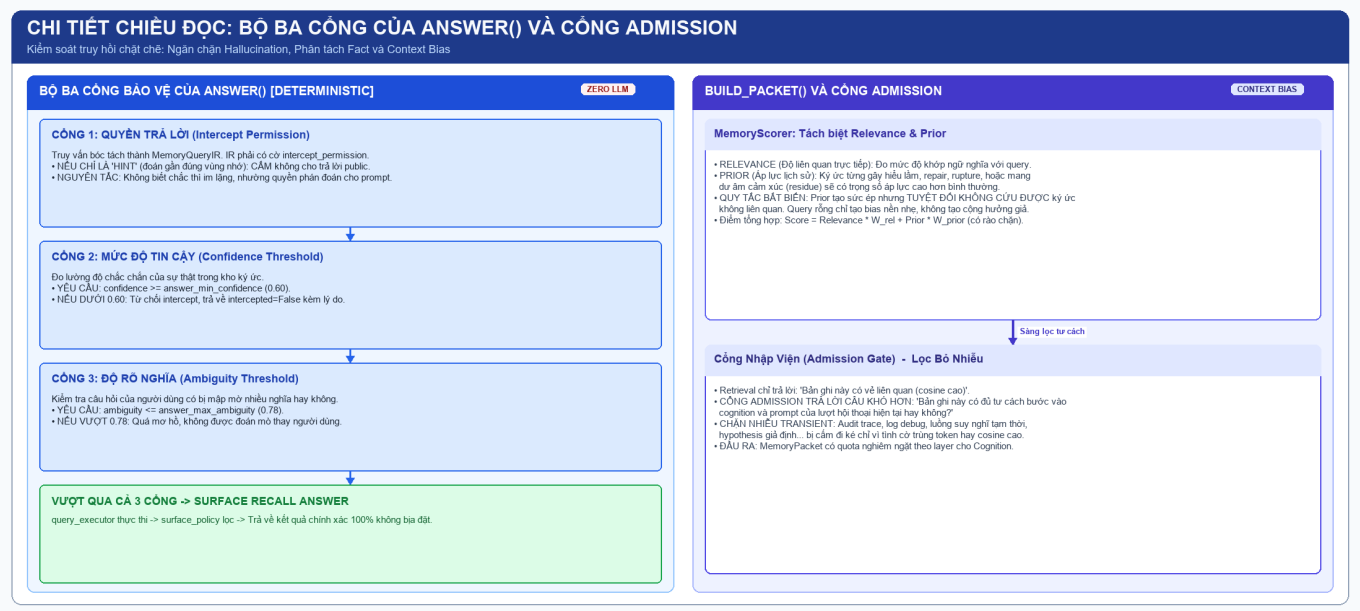
8. Event Ledger (Sổ cái sự kiện)
Cơ chế append-only bất biến, lưu trữ sự thật thô ban đầu: tín hiệu vật lý, evidence ID, frame thô và audit trail. Không suy diễn tâm lý, không merge và không bao giờ chỉnh sửa quá khứ.

9. Canonical Long-Term Stores (Kho nguyên tử dài hạn)
Cấu trúc JSON nguyên tử cho các miền nhận thức đặc thù: belief_store (niềm tin), worldview (thế giới quan), sociocultural (văn hóa xã hội), spatial (không gian), temporal/body rhythm (nhịp sinh học), ToM (Theory of Mind).

[i] NGUYÊN TẮC CỐT LÕI VỀ PHÂN VÙNG LƯU TRỮ
Mỗi kho ký ức được bảo vệ bằng chính sách đọc/ghi riêng. Tuyệt đối không bao giờ trộn lẫn dữ liệu thô (raw events) với dữ liệu đã chưng cất (facts), và không bao giờ để suy đoán cảm xúc (residue) lấn át sự thật khách quan.

5. CHIỀU ĐỌC: ĐỐI CHIẾU ANSWER() VS BUILD_PACKET()

Chiều Đọc của Eris Memory phân tách thành hai con đường hoàn toàn độc lập, giải quyết hai bài toán nhận thức khác biệt: Trả lời sự thật chính xác (answer()) và Cung cấp ngữ cảnh chú ý cho nhận thức (build_packet()):



5.1 answer() - Trả lời Sự thật Deterministic (Zero LLM)

Dành riêng cho các câu hỏi truy vấn trí nhớ cần độ chính xác tuyệt đối (ví dụ: 'tên tôi là gì?', 'tôi sinh tháng mấy?'). Hàm này TUYỆT ĐỐI KHÔNG GỌI LLM để sinh câu trả lời, nhằm loại bỏ 100% rủi ro hallucination. query_compiler phân giải text thành MemoryQueryIR rồi bắt buộc đi qua Bộ Ba Cổng kiểm duyệt nghiêm ngặt:

• Cổng Quyền (Intercept Permission): IR phải có cờ thẩm quyền. Nếu chỉ là 'hint' (đoán gần đúng), cấm cầm mic trả lời.

• Cổng Tin Cậy: Mức độ chắc chắn của sự thật trong kho phải đạt confidence >= 0.60.

• Cổng Rõ Nghĩa: Câu hỏi không được mơ hồ, đòi hỏi ambiguity <= 0.78.

Chỉ khi vượt qua cả ba cổng, query_executor mới chạy và surface_policy mới trả về RecallAnswer. Nếu vi phạm bất kỳ cổng nào, kernel trả về intercepted=False kèm lý do xác thực, nhường quyền phản hồi cho runtime.

5.2 build_packet() - Gói Áp Lực Ngữ Cảnh cho Nhận Thức

MemoryPacket KHÔNG PHẢI là câu trả lời trực tiếp; nó là tập hợp áp lực nội bộ (context bias) được phân bổ theo hạn ngạch (quota) của từng layer, dùng làm context định hướng cho prompt và luồng suy nghĩ của Eris.

• MemoryScorer tách biệt Relevance khỏi Prior: Prior (ký ức từng gây hiểu lầm, repair, rupture hoặc mang dư âm cảm xúc) tạo ra sức ép chú ý lớn hơn, nhưng TUYỆT ĐỐI KHÔNG CỨU ĐƯỢC ký ức không liên quan đến chủ đề hiện tại.

• Cổng Nhập Viện (Admission Gate): Retrieval thông thường chỉ trả lời 'record có vẻ liên quan (cosine cao)'. Admission trả lời câu hỏi cốt tử: 'Record này có đủ tư cách bước vào prompt lượt này không?'. Nhờ đó, rác audit, debug trace, luồng suy nghĩ tạm thời... bị chặn đứng hoàn toàn, không thể đi ké chỉ vì điểm tương đồng vector cao.

Bảng Ma Trận So Sánh: answer() vs build_packet()

Tiêu chí so sánh	answer() (Truy vấn sự thật)	build_packet() (Ngữ cảnh nhận thức)
Mục tiêu cốt lõi	Trả lời chính xác fact đã lưu (tên, ngày sinh...)	Định hình áp lực chú ý & context cho prompt
Sử dụng LLM	TUYỆT ĐỐI KHÔNG (100% Deterministic Code)	Cung cấp ngữ cảnh cho LLM sinh lời thoại
Cơ chế kiểm soát	Bộ Ba Cổng: Quyền, Confidence, Ambiguity	MemoryScorer (Relevance/Prior) & Cổng Admission
Rủi ro Hallucination	Hoàn toàn bằng 0% (Không suy diễn)	Được kiểm soát chặt qua Quota từng layer
Xử lý khi không chắc	Trả intercepted=False, nhường quyền	Hạ điểm ưu tiên, không đưa vào packet

6. LIVING USER MEMORY & VÒNG BẢO TRÌ NỀN

6.1 Living User Memory (24 Modules)

living_user/ là chân dung người dùng 'sống': tổng hợp sở thích, mục tiêu học tập, cam kết cá nhân và quy trình làm việc mà người dùng đã nêu rõ. Runtime chỉ giao tiếp qua Service Facade, không tự ý chọc vào store nội bộ:

• Đặc quyền lời nói: Chỉ lời nói trực tiếp của user (user_statement) mới được đưa vào bộ bóc tách chân dung.

• Portrait Store: Chân dung do Gemini tổng hợp, lưu trữ bền vững trên đĩa nhưng KHÔNG PHẢI sự thật gốc. Khi mở panel, hệ thống chỉ đọc từ đĩa cục bộ, zero network wait. Mỗi bản viết đều có vân tay bằng chứng (evidence fingerprint); nếu nguồn thay đổi (đặc biệt khi user yêu cầu sửa hoặc quên), bản cũ lập tức bị vô hiệu hóa ngay.

• Tách bạch Mastery: Trong ErisTutor, living-user chỉ cấp tín hiệu 'người dùng đã quan tâm / đã nêu', hoàn toàn tách bạch khỏi mức độ nắm vững kiến thức (mastery) do LearningService chứng minh. 'Đã quan tâm' không bằng 'đã làm chủ'.

6.2 Chu trình bảo trì maintain() ở chế độ nền

Chỉ chạy khi hệ thống rảnh rỗi (idle), trong pha reflection hoặc kết thúc phiên (session end), tuyệt đối không bao giờ chen ngang vào hot path chat. MemoryMaintenanceRunner thực thi 4 nhiệm vụ:

1. Consolidate Candidates: Thử nghiệm chưng cất các dấu vết thành fact ổn định (giới hạn budget 6/20/48).
2. Prune Hết Hạn: Dọn dẹp ký ức hết hạn / decay; mặc định an toàn không xóa phá hủy (non-destructive).
3. Contradiction Resolution: Rà quét và liên kết các mệnh đề mâu thuẫn để điều chỉnh trọng số niềm tin.
4. Episode Rollup & Dream: Tóm tắt cuộn sâu và củng cố ký ức dài hạn có độ trễ hợp lý.

7. TÁM BẤT BIẾN CỐT LÕI CỦA HỆ THỐNG (SYSTEM INVARIANTS)

• **1. Model không phải quan tòa:** Nghĩa ban đầu có thể do LLM đọc, nhưng commit, ontology, entailment, worthiness và retention đều do code deterministic quyết.

• **2. Provenance bất biến:** Metadata nguồn gốc (origin_kind, roots, quyền) do Runtime Ingress cấp, mô hình tuyệt đối không thể tự mạo danh ghi đè.

• **3. Một cổng ghi duy nhất:** Mọi hành vi ghi bền vững bắt buộc phải đóng gói thành StateCommitRequest và đi qua MemoryCognitionCore.commit().

• **4. Candidate != Fact:** Dấu vết chưa được chưng cất chỉ nằm ở vùng đệm Unresolved, tuyệt đối không bao giờ được surface ra ngoài như một chân lý.

• **5. Similarity != Admission:** Điểm tương đồng vector cao chỉ là tín hiệu tìm kiếm, không phải giấy thông hành để được đưa vào vùng nhận thức prompt.

• **6. Packet != Answer:** Packet là áp lực định hướng nội bộ cho nhận thức; answer() mới là câu trả lời sự thật và bắt buộc phải qua Bộ Ba Cổng.

• **7. Cảm xúc là áp lực:** Affective residue là áp lực điều hòa nhịp điệu tương tác và khoảng lặng, tuyệt đối không tự động biến thành câu thoại phát ngôn.

• **8. Quyền được quên có hiệu lực thật:** Khi người dùng yêu cầu quên, portrait và bản ghi cũ bị vô hiệu hóa ngay lập tức dựa trên dấu vân tay chứng cứ.

[=] KẾT LUẬN TOÀN DIỆN VỀ KIẾN TRÚC MEMORY ERISTUTOR

Hệ con Memory của ErisTutor thể hiện một tư duy kiến trúc nhận thức trưởng thành: tôn trọng sự thật khách quan, kiểm soát chặt chẽ quyền lực của mô hình ngôn ngữ lớn, và thiết lập các chốt chặn deterministic không thể bị đánh lừa. Chính nhờ sự tách bạch nghiêm ngặt giữa Chiều Ghi, Chiều Đọc, Lưu Trữ Chuyên Biệt và Bảo Trì Nền, ErisTutor đạt được khả năng đồng hành giáo dục bền bỉ, thấu hiểu người học sâu sắc mà không bao giờ rơi vào cái bẫy ảo giác (hallucination).